Final Exam (Part 2)

Program Design and Data Structures (1DL201)

Teachers: Johannes Borgström, Eva Darulova, and Tobias Wrigstad

2023-03-13 / 8:00 - 12:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **14 questions**. Each question can award at most 4 points. You can obtain a maximum of **37 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Johannes and Eva will come to the exam hall around 9:00 to answer questions.

```
/**
 * Converts bottle sizes (volumes) at a particular position in input array
 * from string to numerical format.
 * @precondition ???
 * @example
 * const arr = Array('Mariestads', 'Julebrygd', '330', '500');
 * convert(arr, 2, 3);
 * // contents of arr: [ 'Mariestads', 'Julebrygd', 330, 500 ]
 * @param A array of data
 * @param start starting position of size data to be changed (inclusive)
 * @param end end position of size data to be changed (inclusive)
 */
function convert(A: ???, start: ???, end: ???): ??? {
    for (var i = start ; i <= end ; i++) {
        const elem = A[i];
        const conv = elem === "250" ? 250 : elem === "330" ? 330 : 500;
        A[i] = conv;
    }
}
```

Question 1: Types (3 points)

A bar processes inventory data of its drinks using the function `convert` above. `convert` takes as input an array with textual data corresponding to one particular beverage, and converts the available sizes of bottles from textual to number format for further processing.
Provide the type signature of `convert`, i.e. the types of `A, start, end` and the return value **without using the `any` type**.
Answer:

```
function convert(A: Array<string | number>,
                 start: number,
                 end: number): void {
```

Question 2: Preconditions (2 points)

What is a suitable precondition for the function `convert`? Write `true` if you think the precondition can be omitted.
Answer:
Assumes that bottles have only size 250, 330 and 500.
Justify your answer:
Numbers other than 250, 330 and 500 will not be converted correctly, as the default case of the condition will turn all other size values (including non-numbers) into 500.

Question 3: Stacks (3 points)

Recall the stack data structure and its interface from the lectures:

```
export type NonEmptyStack<T> = [T, Stack<T>];
export type Stack<T> = null | [T, Stack<T>];
export function empty<T>(): Stack<T>
export function is_empty<T>(stck: Stack<T>): stck is null
export function push<T>(e: T, stck: Stack<T>): NonEmptyStack<T>
export function top<T>(stck: NonEmptyStack<T>): T
export function pop<T>(stck: NonEmptyStack<T>): Stack<T>
```

Complete the following Jest testcases for unit testing the stack implementation, **without** using another call to `push`. Make sure that your testing code is correctly typed.

Answer:

```
import { empty, is_empty, pop, push, top } from '../lib/stack';

test('last-in first-out order', () => {
    const stck = push(5, push(7, push(3, empty())));
    expect(top(stck)).toBe(5);
});

test('push pop sequence', () => {
    const stck = push(5, push(7, push(3, empty())));
    const stck2 = pop(stck);
    expect(!is_empty(stck2) && top(stck2)).toBe(7);
});
```

Question 4: Queues (2 points)

Recall the **mutable** implementation of the queue data structure that uses an underlying array to store the queue's elements:

```
// [head index, next empty index, values]
type Queue<T> = [number, number, Array<T>];
```

What is the value of the underlying representation of the queue `que` after the following code?

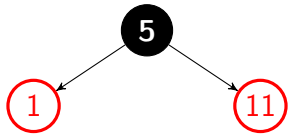
```
const que = empty_queue<number>();
enqueue(5, que);
enqueue(7, que);
dequeue(que);
enqueue(42, que);
dequeue(que);
enqueue(13, que);
```

Answer:

```
[ 2, 4, [ 5, 7, 42, 13]]
```

Question 5: Red-black trees (3 points)

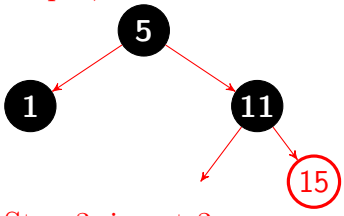
Consider the following red-black tree (we show only the node's keys and ignore the data):



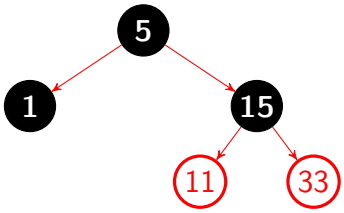
Show the resulting red-black trees after inserting elements with keys 15, 33, and 3, in that order. That is, draw three trees showing the tree after each insertion is completed. Circle red nodes, and leave black nodes without a border (do not use a red pen).

Answer:

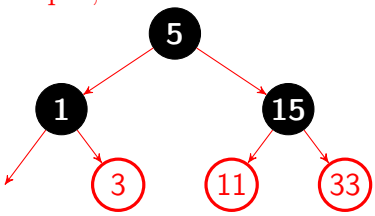
Step 1, insert 15:



Step 2, insert 33:



Step 3, insert 3:



Question 6: Complexity (3 points)

What is the **worst-case time** complexity in terms of Big-Theta of the following operations?

Answer:

- insertion into a red-black tree: $\Theta(log\ n)$
- retrieving an element from a hashtable with n elements that uses chaining: $\Theta(n)$
- topological sort in a directed acyclic graph with V nodes and E edges: $\Theta(|V| + |E|)$

Question 7: Records (2 points)

The binary trees in Source Academy were implemented using the following type:

```
type BinaryTree = null | Pair<Any, Pair<BinaryTree, BinaryTree>>;
```

Rewrite this type using a record type for the non-empty case, letting the type parameter `Value` be the type of the value stored in each node. Use informative names for the fields.

Answer:

```
type BinaryTree<Value> = null | {
                          value: Value,
                          left_child: BinaryTree<Value>,
                          right_child: BinaryTree<Value>
  };
```

Question 8: Heaps (3 points)

Remove the top element from the following array heap using the algorithm seen in class (called MAX-HEAP-EXTRACT-MAX in I2A), showing the contents of the array at each intermediate step.

Answer:

9	5	7	1	3	6	2	(input array)
2	5	7	1	3	6		
7	5	2	1	3	6		
7	5	6	1	3	2		(array after last step)

Question 9: Hashtables (3 points)

Assume that you have a hash table with 4 slots that uses linear probing to resolve collisions. Let $\perp$ denote an empty cell, and Δ a deleted one. Can the hash function $h(s) = (\texttt{length}\ s)^2$ produce the following table?

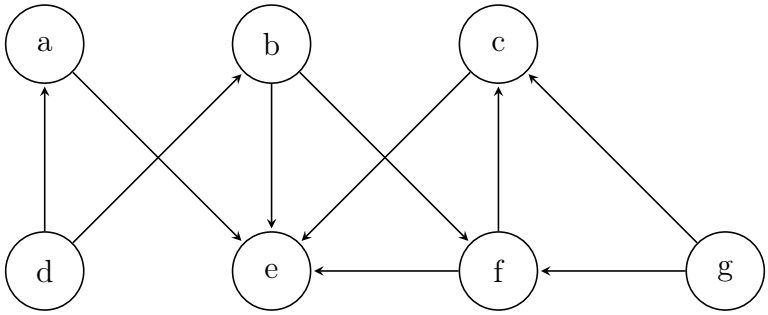
0	1	2	3
fandango	magnifico	Δ	thunder

Answer and Justification:

Yes: Start with an empty hash table. Insert "fandango" at 0, "magnifico" at 1, "lightning" at 2 and "thunder" at 3. Then delete "lightning".

Question 10: Graph representation (2 points)

Consider the following graph:



Give an adjacency matrix representation of the graph.

Answer:

	a	b	c	d	e	f	g
a					1		
b					1	1	
c					1		
d	1	1					
e							
f			1		1		
g			1			1	

Question 11: Graph traversal (3 points)

Consider the graph in question 10. Perform a breadth-first search in the graph, starting at node d. What are its discovery and finish orders (i.e., in which order are the nodes coloured grey and black, respectively)?

Answer:

d	b	a	f	e	c	

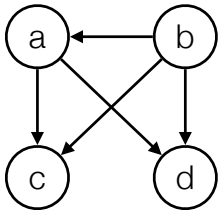
(discovery order)

d	b	a	f	e	c	

(finish order)

Question 12: Topological sort (2 points)

Produce two valid topological sorts of this graph and explain why they are both valid. (Answer format: *a, b, c, d*)



Answer:
Justification:
Solution:
b, a, d, c and b, a, c, d
Justification:
Node b can be the first node in a topological sort, since it has no incoming edges. Node a can follow immediately after b, since it only has an incoming edge from b. Nodes c and d only have incoming edges from a and b, so they can follow a and b in any order.

Question 13: Meta-circular evaluator: semantics of function application (2 points)

Order the following lines so that the resulting code is an application of a function to its arguments. The order of evaluation should precisely match that of JavaScript/Source. (Answer format: *1, 2, 3, 4, 5*)

1. `apply(function_value, list(arg1, arg3, arg2));`
2. `const arg1 = evaluate(exp1, current_environment);`
3. `const function_value = evaluate(function_expression, current_environment);`
4. `const arg3 = evaluate(exp3, current_environment);`
5. `const arg2 = evaluate(exp2, current_environment);`

Answer:
Justify your answer:
Answer: 3, 2, 4, 5, 1
Justification:
The evaluation order of Source/JavaScript is left-to-right, so we start by evaluating the expression that stands for the function we are going to call (line 3), and then evaluate the arguments in left-to-right order (`exp1` (line 2), `exp3` (line 4), `exp2` (line 5)). Finally, we apply the function value to its arguments (line 1).

Question 14: Strongly connected components (4 points)

In a directed graph, we say that the nodes A and B are *weakly connected* if there is a path from A to B or from B to A. If there is a path from both A to B and B to A, we say the nodes are *strongly connected*. (Thus, strongly connected implies weakly connected.)

Write a function `is_weakly_connected(G:ListGraph, n1:number, n2:number)` that returns `true` if there is a path from `n1` to `n2` or from `n2` to `n1` in `G`.

Answer:

```
type ListGraph = {
    adj:  Array<List<number>>; // Lists may not be sorted
    size: number;
}
function is_weakly_connected(G:ListGraph, a:number, b:number): boolean {
    function is_weakly_connected_inner(fro:number, to:number, visited:List<number>): boolean {
        const adj = G.adj[fro];
        function do_it(n:number, found:boolean): boolean {
            return found
                || (!is_null(member(n, visited))
                    && is_weakly_connected_inner(n, to, append(adj, visited)));
        }
        return !is_null(member(to, adj))
            || accumulate(do_it, false, adj);
    }
    return is_weakly_connected_inner(a, b, null)
        || is_weakly_connected_inner(b, a, null);
}
```

Hints:
You may want to use the function `member<T>(v: T, xs: List<T>): List<T>` that takes a value `v` and a list `xs` and returns the first postfix sublist whose head is identical to `v` or `null` if the element is not in the list.

You may want to use a helper function.

This is the final page.